

Ditron s.r.l.

DRIVER WinEcrCom

Lista delle revisioni.

1	Ottobre 2001	Stesura iniziale
2	Giugno 2002	Prima revisione
3		
4		
5		
6		
7		
8		
9		
10		

INDICE DEI CONTENUTI

1. Generalità
2. Terminologia
3. Installazione del driver WinEcrCom
4. Utility di configurazione del driver e definizione delle Porte Logiche
 - 4.1 Esempi
5. Il linguaggio WinEcrCom
6. Struttura dei comandi
 - 6.1 Tipi di istruzioni
7. File di definizione ECR.COM.INI
8. Istruzione "VEND"
9. Istruzione "INP" (Funzione generica)
10. Istruzione EM_LINK
11. Istruzioni di programmazione files
 - 11.1 Istruzione "PROG"
 - 11.2 Istruzione "FINEPROG"
 - 11.3 Istruzione "PREP"
12. Istruzione "LEGGI" (Lettura di files dall'ecr)
13. Traduzione dei messaggi Data-Collection Y (Movimenti)
14. L'oggetto "CoEcrCom" (CO)
15. Codici degli errori
16. Esempio di utilizzo del "Programma di Test/Demo per WinEcrCom"
17. Output Model
18. Proprietà, metodi ed eventi
 - 18.1 Proprietà
 - 18.2 Eventi
 - 18.3 Metodi

1. Generalità

Il driver “WinEcrCom” è un insieme di programmi che consentono il collegamento dei misuratori fiscali in ambiente Windows.

Il driver WinEcrCom permette di stabilire una efficiente “connessione logica” tra un programma “applicativo” e gli ecr collegati al PC. Il driver infatti mette a disposizione un insieme di funzioni che permettono di aprire una connessione, di inviare comandi agli ecr, di gestire gli “eventi” da essi generati e di chiudere la connessione.

Il driver WinEcrCom provvede a rendere del tutto “trasparente” all’applicativo il collegamento fisico (sia esso realizzato in modo diretto in RS232 oppure in rete RS485 mediante l’apposito adapter) ed a “nascondere” la implementazione del protocollo di collegamento e la traduzione di tutti i messaggi scambiati tra programma applicativo e misuratori fiscali, dal formato binario utilizzato dai misuratori stessi, in un formato di tipo testuale di immediata comprensione e di semplice utilizzo. Infatti il driver WinEcrCom implementa uno speciale macro-linguaggio basato su un semplice set di istruzioni che permette di inviare comandi ai misuratori fiscali per sfruttarne appieno tutte le funzioni. E’ possibile, ad esempio, emettere scontrini fiscali dal PC, leggere la programmazione degli ecr, programmare gli ecr, trasmettere e ricevere i files della scheda di espansione di memoria.

Il driver WinEcrCom è uno strumento che è stato progettato per essere utilizzato da chi sviluppa software gestionale nel mondo delle applicazioni retail. Esso quindi deve essere inteso come un insieme di “componenti” che verranno “integrati” all’interno di altri programmi e NON come un programma a sé stante. WinEcrCom è quindi fortemente rivolto ad utilizzatori che abbiano una buona conoscenza dei fondamenti della programmazione in ambiente Windows e che hanno la necessità di “collegare” in modo semplice e veloce i loro programmi applicativi ai misuratori fiscali.

WinEcrCom si basa sul modello “COM”. Si compone di varie parti che verranno successivamente descritte in dettaglio. Sostanzialmente esso si divide in due parti, ossia un “Service Object” ed un “Control Object”. Il “Service Object” è un programma eseguibile che viene automaticamente mandato in esecuzione ogni volta che un programma applicativo richiede l’attivazione di una connessione ai misuratori fiscali. Il Service Object si fa carico di gestire completamente il protocollo fisico di collegamento e la traduzione di tutti i messaggi scambiati. Il “Control Object” è invece un controllo ActiveX, ossia un componente realizzato con una tecnologia standard e che può facilmente essere “inserito” in un programma Windows scritto con un qualsiasi linguaggio di programmazione che supporti la tecnologia ActiveX (Visual Basic, Visual C, Delphi, Builder e molti altri). Il Control Object mette a disposizione dello sviluppatore un set di Metodi, Proprietà ed Eventi che gli consentono di inviare comandi ai misuratori in modo estremamente semplice e rapido.

Una volta compreso il funzionamento del driver WinEcrCom ed appreso come utilizzare il macro-linguaggio da esso implementato, saranno sufficienti pochi click del mouse per aggiungere ai propri programmi applicativi una completa gestione e supporto dei registratori collegati!

2. Terminologia

Nel seguito di questo manuale verranno utilizzati alcuni termini ricorrenti in una forma abbreviata per semplicità. Qui di seguito riportiamo il significato esteso di tali termini.

- Applicativo
E' un programma eseguibile (o un insieme di programmi), realizzato dallo sviluppatore a cui questo manuale è rivolto, all'interno del quale si desidera integrare la possibilità di collegarsi ai misuratori fiscali.
Ovviamente è indispensabile che lo sviluppatore disponga dei "sorgenti" e che sia in grado di aggiungere ad esso le necessarie porzioni di codice per includere le funzionalità messe a disposizione da WinEcrCom.
- SO
E' il "Service Object" ossia quella parte di WinEcrCom costituita dal programma eseguibile "SoEcrCom.EXE". L'SO contiene alcune interfacce che devono essere registrate nel sistema. Tale registrazione viene di norma eseguita dal programma di installazione.
- CO
E' il "Contol Object" ossia il componente ActiveX costituito dal modulo "CoEcrCom.OCX" e che viene incorporato all'interno dell'Applicativo. Anche il CO deve essere registrato nel sistema e tale registrazione viene di norma eseguita dal programma di installazione.
- ECR
Con tale termine indichiamo un misuratore fiscale o registratore di cassa collegato.
- Linguaggio WinEcrCom
E' un insieme di comandi di tipo "testo" che permette di richiamare tutte le funzioni degli ECR all'interno dell'Applicativo. E' molto simile ad un semplice linguaggio di programmazione, ossia si compone di Istruzioni ed Operandi. Il linguaggio WinEcrCom deriva strettamente da quello già utilizzato dall'interprete di comandi "ECRCOM.EXE" per DOS, con il quale risulta quasi del tutto compatibile. La sintassi del linguaggio WinEcrCom è definita dal file "ECRCOM.INI" che, come descritto nel seguito, può anche essere modificato.

3. Installazione del driver WinEcrCom

Nel CD-ROM del driver WinEcrCom si trova un semplice programma di SetUp che deve essere lanciato al fine di eseguire la procedura di installazione del driver.

Tale programma provvederà ad estrarre e copiare tutti i files necessari sul disco rigido del PC. Durante l'installazione, il programma propone la directory :

“C:\Programmi\WinEcrCom”

che può essere liberamente confermata oppure sostituita con un altro nome.

Questa directory, sarà la directory di base nella quale verranno copiati tutti i files necessari e conterrà le seguenti subdirectory ed i seguenti files :

- Subdirectory “DRIVERS”
Contiene i files del driver:
 - SoEcrCom.exe (SO)
 - CoEcrCom.ocx (CO)
 - EcrCom.ini (file di definizione del linguaggio)
 - EcrCom_i.ini (versione italiana di EcrCom.Ini)
 - EcrCom_e.ini (versione inglese di EcrCom.Ini)
 - Prot485.dll (gestione del protocollo fisico in caso di 485)
 - Prot232.dll (gestione del protocollo fisico in caso di 232)
 - EcrTrad.dll (traduttore)
 - Modem.dll (gestione del protocollo fisico in caso di collegamento via Modem)
- Subdirectory “UTILITIES”
Contiene alcuni programmi di utilità, tra cui:
 - WinEcrConf.exe (Utilità di configurazione delle porte logiche)

Inoltre all'interno del CD-ROM si trovano altre due subdirectory :

- Subdirectory “DEMO”
Contiene vari esempi di uso del driver, suddivisi per diversi linguaggi di programmazione, completi dei sorgenti. Contiene anche una subdirectory “ESEMPI” che riporta diversi tipi di files con comandi WinEcrCom.
- Subdirectory “DOC”
Contiene manuali e documentazione tecnica.

Il programma di installazione provvede ad aggiornare il sistema e a registrare i vari componenti. Subito dopo avere installato e registrato i componenti del driver, il programma di installazione lancia automaticamente il programma di configurazione delle porte logiche, che viene descritto nel successivo paragrafo.

4. Utility di configurazione del driver e definizione delle Porte Logiche

Per potere correttamente usare il driver WinEcrCom, è necessario che questo sia correttamente configurato e che siano state definite una o più “Porte Logiche”.

Una “porta logica” definisce in che modo vengono collegati gli ecr al PC. Per ogni porta logica, occorre configurare alcune opzioni che descrivono la connessione.

Le porte logiche sono contraddistinte da un numero (da 1 a N).

Per impostare alcune opzioni di funzionamento di WinEcrCom e per creare, modificare o eliminare una porta logica, occorre utilizzare lo speciale programma “WinEcrConf.exe”. Tale programma verrà automaticamente lanciato al termine della procedura di installazione, ma può essere utilizzato anche successivamente ogni volta che si desidera modificare la configurazione del driver o una porta logica già esistente o se si vuole definire altre porte logiche.

E’ necessario che venga creata almeno una porta logica, altrimenti non sarà possibile aprire il driver WinEcrCom.

Infatti, come descritto successivamente, quando un applicativo vuole aprire una nuova connessione con gli ecr collegati, questo dovrà richiamare il metodo “Open” specificando quale porta logica deve essere usata.

Il programma “WinEcrConf.exe”, per ogni porta logica, chiede di immettere le seguenti informazioni:

- Tipo di connessione fisica. Questa può essere :
 - o RS232
caso di connessione diretta di un solo ecr su una porta seriale del PC
 - o RS485
caso di connessione di un massimo di 12 ecr collegati in rete 485 mediante un adapter 485 installato su una porta seriale del PC
 - o MODEM
caso di connessione di un ecr remoto mediante l’interposizione di una coppia di modems.
- Porta seriale
Specifica la porta seriale del PC usata (COM1, COM2, COM3 oppure COM4).
- Stringa-protocollo ecr
Specifica la stessa stringa-protocollo che viene programmata sugli ecr. Tale stringa definisce :
 - o Velocità di comunicazione (baud-rate)
 - o Bit di parità (solo se RS232 o MODEM)
 - o Numero di stop-bit
 - o Flag di protocollo semplificato (No-Sync)
 - o Flag time-outs lunghi
 - o Flag calcolo checksum

Si consiglia di utilizzare il valore “60185” in caso di RS232 ed RS485.

- Definizione della rete (solo se RS485)
E’ una stringa che definisce quali ecr sono collegati in rete 485. E’ composta da una sequenza di 12 caratteri che possono essere “0” o “1”, dove con “1” si indica che l’ecr in quella posizione è

presente.

Gli ecr sono numerati da 1 a 12, ossia è necessario attribuire gli indirizzi di nodo tra 1 e 12.

Esempi:

Se si vuole collegare solo 4 ecr con gli indirizzi 1,2,3 e 4, occorre specificare il valore "111100000000".

Se si vuole collegare gli ecr agli indirizzi 1,2,6,7,8,9 occorre specificare il valore "110001111000".

- Opzione On-Line

Attivare tale opzione solo se si prevede di gestire i messaggi ecr di accesso a file-esterni e/o data-collect e/o interattività.

Inoltre il programma "WinEcrConf.exe", permette di configurare le seguenti funzionalità generali del driver.

- Opzione "Auto-Run"

Attivare questa opzione se si prevede di utilizzare la speciale modalità di funzionamento "Auto-Run" che consente di fare eseguire un file di comandi WinEcrCom automaticamente con la semplice creazione del file stesso, senza alcun programma specifico.

- File di comando "Auto-Run"

Quando viene attivata la modalità "AutoRun", il programma "WinEcrConf.exe", richiede di specificare il nome (completo del path di ricerca) di un file che il driver provvederà a mandare automaticamente in esecuzione non appena detto file sarà stato creato e salvato da qualsiasi altro programma, ad esempio utilizzando NotePad o un altro text-editor.

Quando la modalità "Auto-Run" è attiva, il driver WinEcrCom,, (più esattamente l'SO), controlla continuamente (ogni secondo) la presenza del file di comando Auto-Run configurato e, non appena lo trova, provvede a mandarlo in esecuzione. Al termine della esecuzione, WinEcrCom rimuove il file dall'hard-disk, scrive l'eventuale file di report degli errori e ritorna quindi a controllare la presenza del file ogni secondo.

- File report errori di "Auto-Run"

Quando viene attivata la modalità "AutoRun", il programma "WinEcrConf.exe", richiede di specificare il nome (completo del path di ricerca) di un file che il driver provvederà a scrivere automaticamente come rapporto degli eventuali errori riscontrati durante l'esecuzione automatica del file di comando Auto-Run.

4.1 Esempi

Mostriamo ora degli esempi di configurazione del driver.

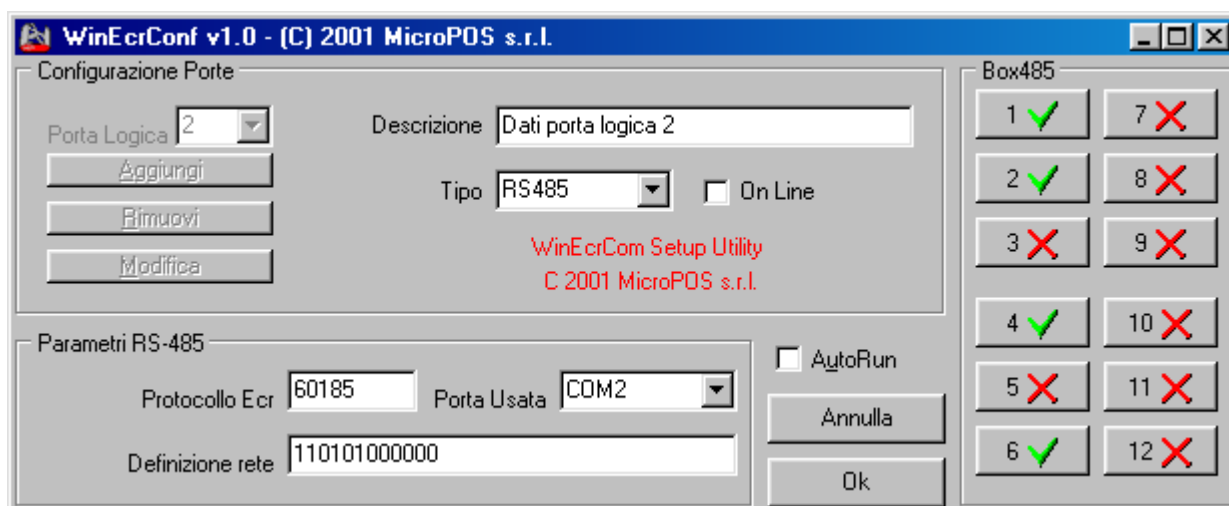
Esempio 1:

Porta Logica: 1
Tipo: RS232
Porta Usata: COM1
Protocollo ECR: 60185
On Line: No
AutoRun: No



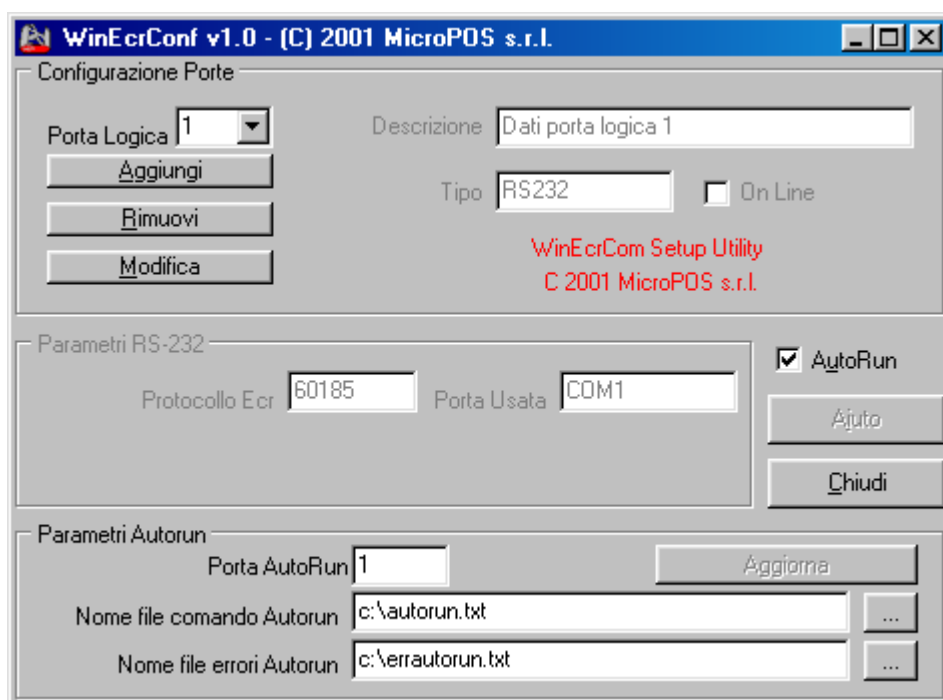
Esempio 2:

Porta Logica: 2
Tipo: RS485
Porta Usata: COM2
Protocollo ECR: 60185
On Line: No
AutoRun: No
Definizione rete: 110101000000



Esempio 3:

Porta Logica: 1
Tipo: RS232
Porta Usata: COM1
Protocollo ECR: 60185
On Line: No
AutoRun: Si
Porta AutoRun: 1
Nome file comando Autorun: c:\autorun.txt
Nome file errori Autorun: c:\errautorun.txt



5. Il linguaggio WinEcrCom

Il driver WinEcrCom definisce una specie di linguaggio di programmazione che consente di inviare comandi agli ecr in modo semplice ed intuitivo.

Il macro-linguaggio WinEcrCom è molto simile a quello adottato dal vecchio programma DOS ECR.COM.EXE, dal quale deriva strettamente. E' possibile continuare ad utilizzare i files usati in precedenza per inviare comandi alle casse mediante ECR.COM.EXE.

La nuova versione del linguaggio estende alcuni comandi. La principale aggiunta è la integrazione con le funzioni del programma DOS EM_LINK.EXE. Infatti è stata aggiunta la nuova istruzione "EM_LINK" mediante la quale è possibile inviare e ricevere gli archivi supportati dalla scheda di espansione memoria (INDIPOS). Con questa nuova istruzione è possibile eseguire tutte le operazioni che venivano in precedenza effettuate utilizzando il programma DOS EM_LINK.EXE.

I comandi definiti dal macro-linguaggio WinEcrCom possono essere direttamente inviati agli ecr mediante i "metodi" messi a disposizione dal driver, oppure è possibile creare un file di testo nel quale scrivere una sequenza di più istruzioni WinEcrCom, ossia un "file lista di comandi o file-comando"; questo file viene quindi passato al driver, il quale scandisce il file, interpreta e traduce i comandi e li trasmette all'ecr in modo che questo li "esegua". Si tratta di una modalità del tipo "Batch".

Il CO di WinEcrCom mette a disposizione il metodo "EcrCmd" che permette di eseguire una singola istruzione oppure un intero file contenente una sequenza di istruzioni. Il driver WinEcrCom provvede ad "interpretare" il comando inviato e lo traduce in una serie di messaggi che vengono trasmessi all'ecr al fine di eseguire la specifica istruzione.

Ogni singolo comando deve essere scritto su una singola linea di testo (in formato ascii) e deve rispettare la sintassi definita dal macro-linguaggio e descritta in questo documento.

E' possibile inviare al driver (con il metodo "EcrCmd") una singola istruzione o un intero file comando.

Per inviare al driver un intero file comando, occorre utilizzare il metodo "EcrCmd" passando a tale metodo una riga di testo nel formato:

"@ nome-file-comando [, nome-file-report-errori]"

ossia una riga che inizia con lo speciale carattere "@" seguita dal nome del file di testo contenente la lista di comandi da eseguire. Opzionalmente è possibile specificare anche il nome di un file nel quale verranno riportati eventuali errori riscontrati durante la esecuzione. Il nome del file report-errori deve essere separato da quello del file di comando mediante l'interposizione di una virgola.

Per ciascuna linea sorgente processata, l'interprete prepara un messaggio di "risultato" che può essere del tipo "OK" se la riga sorgente è stata correttamente riconosciuta, tradotta ed eseguita dall'ecr, oppure può essere un messaggio di "errore" nel caso che l'interprete abbia riscontrato un errore di sintassi nella riga sorgente oppure che l'ecr segnali un errore; tale messaggio di risultato viene restituito dalla funzione "EcrCmd" in modo che il programma applicativo può verificare la corretta esecuzione del comando.

Nel caso di esecuzione batch di un intero file, gli eventuali errori generati dalla esecuzione di ogni singola linea sorgente, vengono accodati sul file “report errori” che è stato opzionalmente specificato insieme al nome del file comando. Per ogni errore riscontrato verrà indicato il numero della linea sorgente al quale si riferisce.

Avvalendosi della speciale modalità di funzionamento “Auto-Run” è possibile configurare il driver in modo che questo provveda automaticamente ad eseguire un determinato file comando non appena il driver “trova” detto file nella posizione configurata. Al termine della esecuzione, il file sorgente viene poi automaticamente cancellato. In tal caso non è più necessario integrare all’interno di un applicativo il CO ed utilizzare il metodo “EcrCmd” per mandare in esecuzione il file. Basterà semplicemente creare un file con i comandi che si desidera inviare alle casse e poi copiare o ridenominare tale file, con il nome (completo di path) che è stato configurato come “File comando di Auto-Run”.

Il driver stesso riconoscerà l’esistenza del file e lo eseguirà automaticamente.

6. Struttura dei comandi

La struttura generale di un “comando” è la seguente :

Istruzione operando, operando, ;commento

Istruzione :

E' un codice operativo che identifica il comando stesso ossia stabilisce quale azione deve essere intrapresa dall'ecr.

L'interprete trasforma in maiuscole le lettere minuscole usate per *istruzione* ossia non c'è distinzione tra maiuscole e minuscole.

Operando :

E' in generale una espressione che definisce il “valore” di un “campo”.

Tra *istruzione* e il primo *operando* deve essere inserito almeno uno spazio di separazione; i vari *operandi* devono essere separati tra loro inserendo una virgola “,”.

Gli operandi possono essere immessi in qualsiasi ordine e possono essere inseriti degli ulteriori spazi di separazione.

In generale gli operandi sono “facoltativi” tranne in alcune istruzioni che prevedono alcuni operandi obbligatori.

Commento :

E' una stringa libera che non viene considerata dall'interprete e che può essere immessa al fine di documentare e commentare la linea. Il *commento* deve essere preceduto da un carattere di punto e virgola “;”.

La struttura generale degli operandi è del tipo :

identificativo-campo = valore oppure semplicemente
identificativo-campo

identificativo-campo :

E' una parola-chiave che definisce di quale campo si intende specificare il valore.

L'interprete trasforma in maiuscole le lettere minuscole usate per *identificativo-campo* ossia non c'è distinzione tra maiuscole e minuscole.

valore :

E' il valore che si vuole attribuire a quel determinato campo.

Se il *valore* viene omesso, viene in generale attribuito il valore “VERO” al campo specificato ossia si sottintende che il campo è di tipo “logico” ossia del tipo VERO o FALSO.

Tra *identificativo-campo* e *valore* deve essere inserito un segno di uguale “=”, eventualmente circondato da spazi.

In generale i campi prevedono un valore che può essere numerico oppure alfanumerico.

Se *valore* è numerico, questo deve essere immesso usando solo cifre e l'eventuale punto decimale per specificare cifre decimali. Se il numero è negativo si deve premettere un carattere “-” prima del numero.

Esempi di valori numerici sono :

100
25.5
-10

Se *valore* è alfanumerico, questo deve essere immesso racchiudendolo tra una coppia di apici (carattere “”) e deve essere costituito da una stringa di caratteri ascii.

Se nella stringa fosse presente un carattere “”, occorre prefissarlo con un altro carattere “”.

Esempi di valori alfanumerici validi sono :

‘PIZZA MARGHERITA’
‘CAFFE’
‘PAPA’ E MAMMA’

Il *valore* può anche essere specificato usando un simbolo ossia una etichetta che viene poi automaticamente rimpiazzata dal valore numerico (o dalla stringa) assegnata al simbolo usato.

Ad esempio alcuni campi prevedono il valore “1” per indicare che quel campo è posto a “NO” oppure il valore “2” per indicare che è posto a “SI”. In tali casi è possibile specificare il simbolo “SI” che viene tradotto in “2” oppure il simbolo “NO” che viene tradotto in “1”.

Esempio di linea sorgente :

VEND REP=3, PREZZO=1500, DES='COCACOLA' ;Vendita su reparto 3

In questa linea l'istruzione è “VEND” e sono stati immessi tre operandi :

REP = 3	<i>identificativo-campo</i> = “REP”	<i>valore</i> = “3”
PREZZO = 1500	<i>identificativo-campo</i> = “PREZZO”	<i>valore</i> = “1500”
DES = ‘COCACOLA’	<i>identificativo-campo</i> = “DES”	<i>valore</i> = “‘COCACOLA’”

E’ stato anche immesso un commento preceduto dal carattere “;”

Le *istruzioni* e gli *identificativi-campo* possono essere immessi anche usando più o anche meno caratteri di quelli previsti.

Ad esempio la istruzione “VEND” può essere scritta anche come “VENDITA” e verrà interpretata come “VEND”.

Inoltre può anche essere specificata come “VE” o anche “V” ammesso che non esistano altre istruzioni che hanno gli stessi caratteri iniziali, nel qual caso l’interprete segnalerà errore di sintassi per parola-chiave non univoca.

Riferendoci all’esempio suddetto, la linea può anche essere scritta come :

VENDITA R=3,P=1500,D='COCACOLA' oppure come
VEND Reparto=3, PrezzoUnitario = 1500, Descrizione = 'COCACOLA'

6.1 Tipi di istruzioni

Sono previsti i seguenti “tipi” di istruzioni :

- istruzioni di controllo dell'ecr
- istruzioni per la compilazione di uno scontrino (fiscale) di vendita
- istruzioni per la programmazione dell'ecr da PC
- istruzioni per la lettura dei dati contenuti nella memoria dell'ECR
- speciale istruzione “EM_LINK”, per trasmettere e/o ricevere gli archivi supportati dalla scheda di espansione memoria.

7. File di definizione ECRCOM.INI

La sintassi dei comandi è definita nel file ECRCOM.INI che deve trovarsi nella sotto-directory “DRIVERS” della directory di installazione del driver WinEcrCom.

Questo file è suddiviso in sezioni identificate da un nome-sezione racchiuso tra parentesi quadre.

E’ possibile modificare a piacimento tale file e quindi è possibile assegnare nomi simbolici diversi sia alle *istruzioni* che agli *identificativi-campo*, così come è possibile creare dei nomi simbolici “alternativi” detti “alias” in modo che, ad esempio, la istruzione “VEND” possa essere riconosciuta dall’interprete anche come “SALE”, ossia “vendita” in inglese.

La prima sezione [OPCOD] definisce i vari codici-operativi del linguaggio associando un numero d’ordine ad ogni nome simbolico delle *istruzioni*. Non è possibile modificare i numeri d’ordine delle varie istruzioni ma è possibile modificare il nome simbolico.

Seguono delle sezioni che definiscono, per ogni *istruzione*, gli *operandi* previsti per quella istruzione.

Consultando tale file si evince facilmente quali sono le *istruzioni* implementate e i vari *operandi* previsti.

Seguono poi delle sezioni di definizione di labels simboliche; in particolare la [EQUASTI] definisce dei nomi simbolici per i codici-tasto dell’ecr.

Ci sono anche delle sezioni che definiscono i messaggi di errore che l’interprete restituisce ed una sezione che definisce i vari messaggi di testo usati dal driver. E’ facilmente possibile, quindi, modificare tali messaggi per adattarli a lingue diverse.

Il programma di installazione WinEcrCom, copia automaticamente due diverse versioni di ECRCOM.INI, ossia :

- ECRCOM_I.INI = Versione in lingua italiana
- ECRCOM_E.INI = Versione in lingua inglese

Il file ECRCOM_I.INI viene copiato anche con il nome ECRCOM.INI, ossia è quello usato per default. Per utilizzare la versione in Inglese basta ricopiare il file ECRCOM_E.INI sul file ECRCOM.INI.

Il file ECRCOM.INI viene fornito completo di commenti che descrivono tutte le sue parti.

Leggendo il file si ottengono facilmente informazioni sui vari comandi implementati.

Inoltre, nella subdirectory DEMO\ESEMPI, vengono forniti diversi esempi di files sorgenti con comandi WinEcrCom che illustrano la maggior parte delle istruzioni implementate.

Qui di seguito, in questo manuale, verranno descritti solo alcuni dei comandi esistenti (tra cui “VEND” ed “EM_LINK”) a titolo di esempio, rimandando alla lettura del file ECRCOM.INI e agli esempi contenuti in DEMO\ESEMPI per tutti gli altri comandi.

8. Istruzione “VEND”

Questa istruzione serve per attivare una vendita che può essere sia su “reparto” che su “plu”. La sintassi completa è :

VEND REP=*numero-reparto*, ART=*codice-articolo*, PREZZO=*prezzo*, DES=*descrizione*,
QTY=*numero di pezzi*, STORNO, RESO

REP	specifica il numero del reparto, in caso di vendita su “reparto”
ART	specifica il codice-articolo da vendere in caso di vendita su “plu”
PREZZO	specifica il valore del prezzo unitario di vendita; se viene omesso l’ecr applica il prezzo programmato per il dato reparto/articolo
QTY	specifica la quantità ossia il numero di pezzi da vendere; se omesso l’ecr assume quantità unitaria
DES	specifica la descrizione simbolica da stampare sullo scontrino, ossia il nome del reparto/articolo; se omessa, l’ecr stampa la descrizione programmata per il reparto/articolo
RESO	se presente, specifica un reso-merce ossia rende negativa la vendita
STORNO	se presente, specifica che si vuole stornare tale item annullando una vendita precedente.

9. Istruzione “INP” (Funzione generica)

Con questa istruzione è possibile inviare ad un ecr un generico comando in “Emulazione Tastiera” ed è quindi possibile usare tale istruzione per eseguire qualsiasi funzione dell’ecr.

L’istruzione “INP” è del tipo :

INP NUM = *input numerico*, ALFA = *input alfanumerico*, TERM = *codice funzione*

NUM = *input numerico*

Specifica l’eventuale valore numerico di input

ALFA = *input alfanumerico*

Specifica l’eventuale valore alfanumerico di input

TERM = *codice funzione*

Specifica il codice della funzione richiesta.

In generale l’ecr accetta in input un valore numerico ed un tasto di funzione (anche detto “terminatore”). Per alcune funzioni è previsto anche un input “alfanumerico”.

Ogni “terminatore” è identificato da un codice funzione. Ad esempio la funzione “REPARTO 1” ha codice 75.

Quindi, ad esempio, per richiamare una vendita di 10 € al reparto 1, basta preparare la seguente istruzione :

INP NUM=10, TERM = 75.

Nel file ECR.COM.INI, il paragrafo [EQUASTI] definisce dei valori simbolici per tutti i tasti funzione dell’ecr (Labels). Quindi la istruzione suddetta si può anche scrivere come:

INP NUM=10, TERM = DPT1

L’interprete di comandi di WinEcrCom sostituisce il valore 75 alla costante simbolica DPT1 e poi esegue l’istruzione.

10. Istruzione EM_LINK

Questa istruzione serve per eseguire l'upload di files verso la scheda di espansione memoria installata sugli ecr e il download di files da tale scheda.

Il formato dei dati è lo stesso di quello usato dal programma EM_LINK.EXE per DOS.

Prima di usare il comando EM_LINK occorre inviare il comando "SELEZ ECR=n" al fine di selezionare l'ecr verso il quale si desidera effettuare il trasferimento.

EM_LINK CMD = *comando*, FILE = *file dati di input/output*, ERR=*file di errori*,
RESET, CLEAR, BLOCK, APPEND, INFO,
SELEZ = *criterio di selezione articoli*, SELDPT = *numero reparto*,
NUMSCO = *numero scontrino*, DATASCO = *data scontrino*,
TRAD, OFFLINE

Segue una descrizione dei vari operandi.

CMD : alfanumerico

Indica il codice comando, che può essere:

'UA' = Upload Articoli	Trasmette il file articoli all'ecr
'DA' = Download Articoli	Riceve il file articoli dall'ecr
'UO' = Upload Offerte	Trasmette il file offerte all'ecr
'DO' = Download Offerte	Riceve il file offerte dall'ecr
'DM' = Download Movimenti	Riceve il file data-collection Y (movimenti)
'RM' = Azzera il file movimenti	
'DT' = Download Totali TG	Riceve il file con i totali generali
'DF' = Download Totali Fiscali	Riceve il file con i totali fiscali del giorno

FILE : alfanumerico

Specifica il nome completo del file di dati.

Tale nome indica il file che contiene i dati da trasmettere, nel caso di un comando di up-load ('UA', 'UO').

Indica il nome del file nel quale verranno accodati i dati ricevuti, nel caso di un comando di down-load ('DA', 'DO', 'DM', 'DT', 'DF').

ERR : alfanumerico

Specifica il nome completo di un file nel quale verranno accodati gli eventuali errori riscontrati durante la esecuzione del comando.

RESET

Questa opzione richiede la totale cancellazione dell'archivio articoli quando si effettua un comando di tipo 'UA'.

CLEAR

Questa opzione richiede di azzerare i campi "Quantità Venduta" e/o "Valore Venduto" quando si effettua un comando "DA".

BLOCK [= n]

Serve per richiedere il temporaneo "blocco" dell'ecr durante la esecuzione di un comando. E' possibile specificare opzionalmente un valore numerico che può essere :

BLOCK = 1 : blocco dell'ecr (equivale a indicare solo BLOCK)

BLOCK = 2 : se la transazione è aperta sull'ecr (ossia è in corso una vendita) il driver attende che la transazione venga chiusa e solo a questo punto "blocca" l'ecr ed esegue il comando richiesto. Durante l'attesa che la transazione si chiuda, il driver invia continuamente l'evento "Progress" con apposito messaggio per consentire all'applicativo l'eventuale abort della operazione.

APPEND

Questa opzione richiede di aprire il file di dati in output in modalità "append" ossia accodando i dati letti a quelli già eventualmente contenuti nel file specificato.

Quando l'opzione APPEND non viene usata, il file dati specificato viene in ogni caso ricreato di nuovo, sovrascrivendo eventuali dati in esso contenuti.

INFO

Specificando questa opzione, il driver aggiunge alcune righe di commento (ossia che iniziano con un carattere ";") sul file di uscita in caso di comandi di down-load. Tali commenti riportano informazioni sul comando richiesto, l'indirizzo di nodo dell'ecr selezionato e la sua matricola e versione, la data e l'ora e l'esito della operazione.

SELEZ = n

Questo valore determina un criterio di selezione dei records articolo che saranno prelevati con un comando 'DA'. Il valore "n" può essere:

0 = preleva tutti i campi di tutti records presenti in archivio

1 = preleva solo i campi Codice, Giacenza, Qty, Valore di tutti i records

2 = preleva tutti i campi dei soli records "movimentati"

4 = preleva solo i campi Codice, Giacenza, Qty, Valore dei soli records "movimentati".

SELDPT = r

Questo valore determina un criterio di selezione dei records articolo in base al reparto di appartenenza. Il valore "r" indica il numero di reparto per il quale si vuole prelevare gli articoli.

NUMSCO = n

Questo valore determina un criterio di selezione dei records data-collection (archivio "movimenti"). Consente di leggere (NOTA : senza rimuovere) solo i movimenti relativi ad un dato numero di scontrino.

Specificando NUMSCO = 0, verranno letti tutti i movimenti presenti nella memoria, senza però rimuoverli da essa.

DATASCO = 'GGMMAA' (alfanumerico)

Con questo operando è possibile specificare anche la data (GGMMAA) di estrazione dei movimenti (data-collection). NOTA: in tal caso i records estratti NON saranno rimossi dalla memoria dell'ecr.

TRAD

Questa opzione richiede di effettuare la “traduzione” dei records movimenti (data-collection) dal formato interno dell'ecr al formato esportato dal driver WinEcrCom per i messaggi data-collection Y (vedere paragrafo “Traduzione dei messaggi data-collection”).

OFFLINE

Questa opzione può essere usata nel caso di connessione in rete 485 di più ecr, quando risulti attivata la modalità On-Line.

Specificando la opzione OFF-LINE, il driver “sospende” la continua scansione di tutte le casse collegate per aumentare la velocità di esecuzione del comando ossia disabilita temporaneamente la modalità On-Line.

Il formato dei dati utilizzato per gli archivi Articoli ed Offerte è lo stesso di quello implementato nel vecchio programma DOS EM_LINK.EXE. Quindi è possibile utilizzare gli stessi files.

Per quanto riguarda i file in output è possibile modificare alcuni criteri di allineamento dei campi mediante la proprietà “OutEditOptions” (vedere il paragrafo “Proprietà”).

Esempi :

La subdirectory DEMO\ESEMPI, presente nel disco di installazione di WinEcrCom, contiene anche alcuni esempi di utilizzo della istruzione EM_LINK.

11. Istruzioni di programmazione files

WinEcrCom mette a disposizione alcune istruzioni che permettono di programmare la cassa da PC. La cassa in generale dispone di alcuni files di dati programmabili. Tipicamente esistono i seguenti files :

- 1 = Reparti
- 2 = Plu
- 3 = Iva
- 4 = Intestazione
- 5 = Modificatori (percentuali A e B)
- 6 = Opzioni
- 7 = Valute estere
- 8 = Gruppi
- 9 = Subtenders

Per ciascuno di tali files è stata definita una specifica istruzione e gli operandi di tali istruzioni definiscono i valori da programmare sui vari campi. La programmazione avviene utilizzando il meccanismo della emulazione-tastiera, ossia l'interprete di comandi traduce tali istruzioni nelle relative sequenze di tasti normalmente previste per la programmazione fatta manualmente agendo sulla tastiera del registratore. Ciò consente di simulare esattamente la programmazione fatta agendo sulla tastiera dell'ecr.

Gli operandi delle istruzioni di programmazione files sono in generale facoltativi: se un campo non è specificato, l'interprete genera una sequenza che non modifica quel campo ossia lascia il valore corrente, altrimenti lo modifica. Dato che viene simulata la sequenza manuale di programmazione, il registratore può stampare il normale scontrino di programmazione e si ha quindi un riscontro cartaceo delle programmazioni effettuate; la stampa può comunque essere disabilitata.

11.1 Istruzione "PROG"

Questa istruzione va utilizzata per attivare la modalità di "programmazione files".

Essa predispone l'ecr in chiave SET e predispone l'interprete a eseguire le successive istruzioni di programmazione di files su ecr, finché non verrà trovata la speciale istruzione "FINEPROG" che termina la modalità "programmazione files".

La sintassi completa è :

PROG NOPRINT

L'operando NOPRINT è facoltativo ed è di tipo logico ossia non è necessario immettere alcun valore.

Se viene specificato NOPRINT, si istruisce l'ecr a non stampare nulla durante le successive istruzioni di programmazione di files mentre se non viene specificata, l'ecr stamperà normali scontrini di riepilogo delle programmazioni eseguite.

NOTA : L'istruzione PROG deve essere obbligatoriamente immessa prima della prima istruzione di programmazione files su ecr.

11.2 Istruzione “FINEPROG”

Questa istruzione termina la speciale modalità di programmazione di files sulla cassa.

Non prevede alcun operando.

NOTA : L’istruzione FINEPROG deve essere obbligatoriamente immessa dopo l’ultima istruzione di programmazione files su ecr.

11.3 Istruzione “PREP”

Questa istruzione serve per programmare un reparto e va usata solo dopo avere aperto la modalità di programmazione files su ecr.

La sintassi completa è :

PREP NR=*numero-reparto*, DES=*descrizione*, PREZZO=*prezzo*, LIS=*numero-cifre-max*,
IVA=*codice-iva*, BAT=*flag-battuta-singola*, GRU=*codice-gruppo*

NR	specifica il numero del reparto da programmare ed è obbligatorio
DES	specifica la descrizione alfanumerica da attribuire al reparto
PREZZO	specifica il valore del prezzo di base del reparto
LIS	specifica la listing-capacity ossia il max numero di cifre ammesso in input su quel reparto
IVA	specifica il codice dell’aliquota iva da assegnare al reparto
BAT	specifica se si vuole o no la funzione di battuta singola su quel reparto ed il valore da assegnare deve essere 1 oppure NO se non si vuole, altrimenti deve essere 2 oppure SI se si vuole la battuta singola
GRU	specifica il codice del gruppo di appartenenza

12. Istruzione “LEGGI” (Lettura di files dall’ecr)

Tale istruzione consente di leggere un file dall’ecr. La sua sintassi è :

LEGGI NF=*numero-file*, FILE=*nome-file*, SETPROG, APPEND

La descrizione dei vari operandi è :

NF	Numero del file da leggere
FILE	Nome del file dove salvare i dati letti
APPEND	Specifica se deve accodare i dati sul file
SETPROG	Include le linee "PROG" / "ENDPROG"

13. Traduzione dei messaggi Data-Collection Y (Movimenti)

I messaggi “data-collection Y” sono trasmessi dagli ecr collegati ogni volta che viene effettuata una qualsiasi funzione oppure sono memorizzati nel file “movimenti” presente sulla scheda di espansione memoria ed in tal caso, possono essere prelevati con un comando del tipo “EM_LINK CMD='DM', FILE=..... “ (download movimenti).

In ogni caso i records data-collection possono essere “tradotti” da WinEcrCom in modo da poterli esportare in un formato quanto più possibile indipendente dal particolare ecr da cui sono stati generati e che ne rende molto più semplice l’analisi da parte di un generico programma applicativo che voglia “estrarre” i dati di vendita con il desiderato livello di dettaglio.

Per ogni record tradotto, viene generata una riga di tipo testo con una serie di campi separati tra di loro dal carattere di separazione impostato, che per default è la virgola.

Il primo campo è un codice-funzione, ossia un valore numerico che identifica la specifica funzione eseguita dall’ecr. I campi che seguono dipendono dal codice-funzione. Sono previsti sia campi numerici che alfanumerici, i quali sono racchiusi tra una coppia di apici.

I campi numerici possono essere con segno, come nel caso di importi e quantità, oppure senza segno, come nel caso di codice-articolo o numero operatore.

I valori numerici sono sempre riportati senza l’interposizione del punto decimale anche per i campi che prevedono cifre decimali, ossia si utilizza il sistema della “virgola fissa”.

Ad esempio tutti gli importi vanno sempre considerati con il numero di decimali previsti (due decimali fissi in caso di Euro) e le quantità vendute vanno considerate con tre decimali fissi.

Il modo con cui sono formattati i campi numerici, dipende anche dal valore che è stato impostato per la Proprietà “OutEditOptions”, per la quale si rimanda all’apposito paragrafo.

Il codice-funzione viene sempre riportato sui primi due caratteri della linea. Esso quindi può variare da “01” a “99”.

Segue una breve descrizione dei codici-funzione implementati. Alcuni campi sono “opzionali” ossia possono esserci oppure no e questi sono evidenziati nel testo che segue con una coppia di parentesi quadre. Con “ND” si intende il numero di decimali della valuta base dell’ecr.

02 : Vendita a reparto

02, rep, qty, valore

rep	Numero di reparto
qty	Quantità venduta, con segno e tre decimali fissi
valore	Importo totale della vendita, con segno e ND decimali fissi

03 : Vendita su articolo

03, codice, rep, qty, valore [,cod.offerta, sconto]

codice	Codice dell'articolo venduto; può essere numerico oppure alfanumerico nel qual caso viene racchiuso tra una coppia di apici
rep	Reparto di appartenenza
qty	Quantità venduta, con segno e tre decimali fissi
valore	Valore totale della vendita, con segno e ND decimali fissi
cod.off.	Eventuale codice offerta speciale applicata alla vendita
sconto	Eventuale valore dello sconto totale applicato in base all'offerta. Campo con segno e ND decimali fissi

04 : Chiusura transazione

04, time-stamp, num.sco, totale, [bollini]

time-stamp Stringa data e ora (alfanumerico) nel formato 'GG/MM/AA HH:MM'

num.sco	Numero dello scontrino chiuso (emesso)
totale	Totale dello scontrino, senza segno e con ND decimali fissi
bollini	Eventuale numero di bollini-punto assegnati

05 : Non Add Key

05, numero

numero	Numero che è stato digitato dall'operatore
--------	--

06 : Cambio Operatore

06, cod.ope

cod.ope	Codice numerico (numero) dell'operatore che è stato immesso ed accettato
---------	--

07 : Pagamento (valore dato)

07, tender, valore [,tender, valore] [,tender, valore]....

tender	Codice del tipo di pagamento (codice-tender) selezionato
valore	Importo dato

08 : Cambio chiave assetto operativo

08, num.chiave

num.ch. Numero della chiave assetto operativo selezionata

09 : Sconto/Maggiorazione % su item

09, perc, valore, [codice]

perc	Valore della percentuale applicata, con due decimali fissi
valore	Importo dello sconto/maggiorazione, con segno e ND decimali fissi
codice	Eventuale codice dell'articolo nel caso in cui lo sconto sia stato applicato ad un articolo. Può essere numerico o alfanumerico

10 : Sconto/Maggiorazione % su Subtotale

11 : Sconto assoluto su item

11, valore, [codice]

valore	Importo dello sconto, con segno e ND decimali fissi
codice	Eventuale codice dell'articolo nel caso in cui lo sconto sia stato applicato ad un articolo. Può essere numerico o alfanumerico

12 : Sconto assoluto su Subtotale

12, valore

valore	Importo dello sconto, con segno e ND decimali fissi
--------	---

14 : Subtotale

16 : Chiusura transazione su slip-printer

17 : Enquiry Chip-card

18 : Update Chip-card

19 : Set Listino

20 : Offerta

21 : Maggiorazione assoluta su item

21, valore, [codice]

valore	Importo della maggiorazione, con segno e ND decimali fissi
codice	Eventuale codice dell'articolo nel caso in cui lo sconto sia stato applicato ad un articolo. Può essere numerico o alfanumerico

22 : Maggiorazione assoluta su Subtotale

22, valore

valore	Importo dello sconto, con segno e ND decimali fissi
--------	---

30 : Sconto speciale "Entrance Ticket"

14. L'oggetto "CoEcrCom" (CO)

Il Control Object (CO) messo a disposizione dal driver WinEcrCom, è l'oggetto ActiveX che può essere facilmente "incorporato" in un generico programma applicativo, al fine di stabilire una efficiente connessione logica tra l'applicativo stesso e gli ecr collegati.

Il file "CoEcrCom.ocx" contiene l'oggetto da includere negli applicativi. Tale file viene automaticamente "registrato" sul sistema dal programma di installazione di WinEcrCom.

Come ogni oggetto ActiveX, il CO mette a disposizione del programmatore un set di Metodi, Proprietà ed Eventi.

Di seguito viene fornita una breve descrizione di tali Metodi, Proprietà ed Eventi. Nella subdirectory DEMO del disco di installazione del driver sono forniti alcuni esempi di programmazione in VisualBasic e C++ di utilizzo del CO, ai quali si rimanda per una descrizione più dettagliata e pratica.

15. Codici degli errori

Gli errori eventualmente rilevati da WinEcrCom durante l'esecuzione dell'operazione richiesta, vengono elencati nel file degli errori (default ERR.OUT).

Inoltre essi sono riportati a video in forma riassuntiva, ossia al termine dell'esecuzione viene riportato il tipo di errore che fa riferimento alla seguente classificazione :

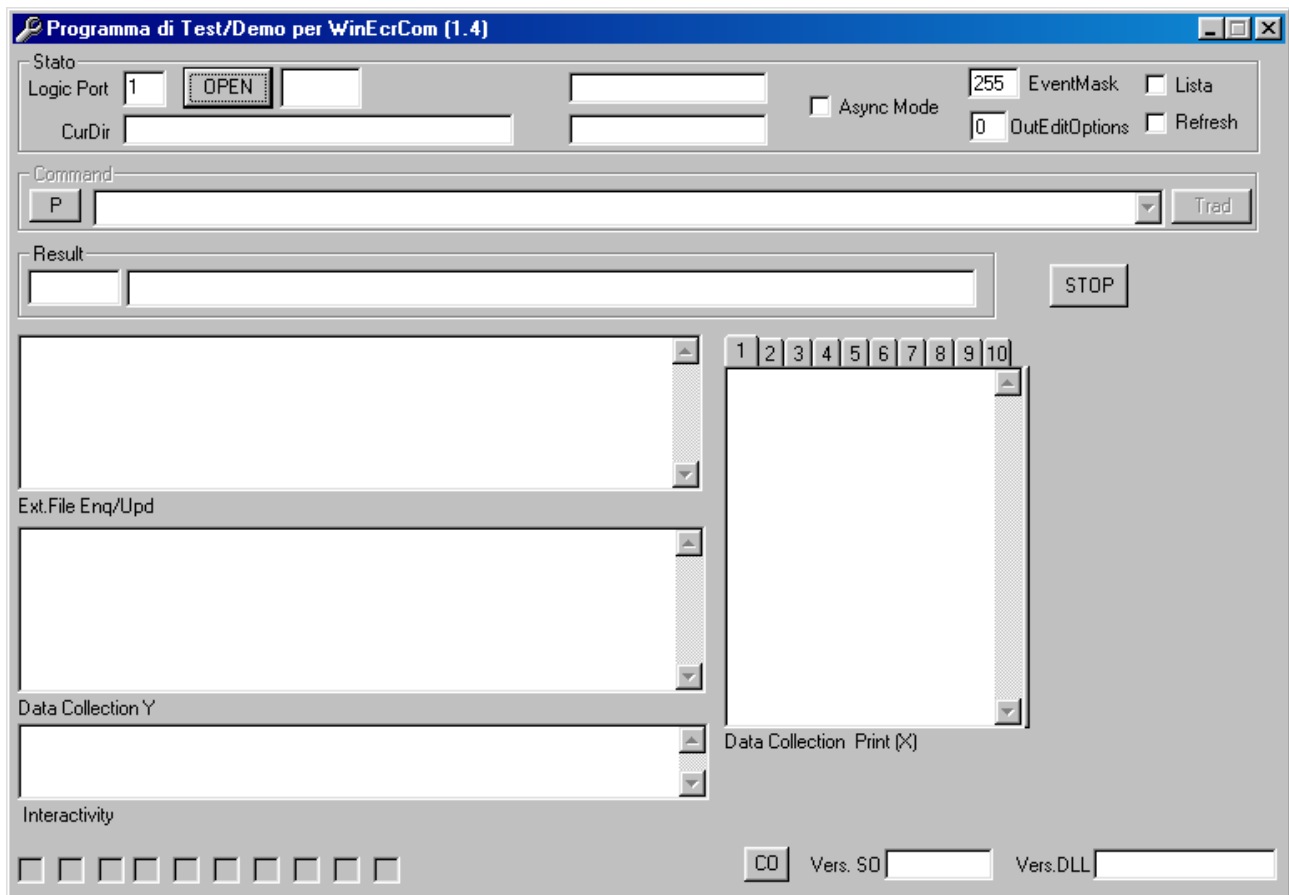
TIPO	CODICE	DESCRIZIONE
0 (BASE)	1	Driver non aperto (cioè non è stato aperto l'SO)
	2	Traduttore non inizializzato
	3	È aperta una sessione di programmazione
	4	Impossibile leggere la configurazione dell'ECR
	5	ECR con versione incompatibile
	6	L'ECR non è in Idle
	7	Sessione di programmazione non aperta
	8	L'ECR non si trova in chiave REG
	9	Scheda di espansione memoria non inizializzata
	10	Linea batch non valida
	11	Errore in apertura del file di input
	12	Errore in apertura del file degli errori
	13	Errore in apertura del file di log
	14	Errori durante l'esecuzione del batch (cioè del file ECRCOM)
	15	Transazione non aperta
	16	Errori durante l'esecuzione
	17	Errore di apertura dell'SO (cioè driver non trovato o non registrato)
	18	Driver già aperto
	19	Driver occupato (cioè comando in esecuzione)
	20	Porta logica non disponibile o non configurata
	21	Errore nel settaggio di una proprietà
	22	Parametri non validi sulla stringa opzioni nella OPEN
	23	Proprietà non valida (cioè errore su istruzione SETP)
	24	Proprietà non valida (cioè errore su istruzione GETP)
	25	Errore chip-card
	26	Errore chip-card non vergine
	27	Nessun ECR selezionato
	28	Time-out connessione modem
	29	Errore modem
	30	Errore in trasmissione broadcast (SPECIALE)
1 (SINTASSI)	0	Errore di sintassi
	1	Istruzione non trovata
	2	Istruzione non univoca
	3	Istruzione illegale
	4	Operando non trovato
	5	Operando non univoco
	6	Operando illegale

	7	Costante non trovata
	8	Valore alfabetico non valido (apici)
	9	Valore illegale
	10	Operando obbligatorio non immesso
	11	Fine linea imprevisto
	12	Istruzione non supportata da questo traduttore
	13	Manca REPARTO o ARTICOLO su VEND
	14	Numero reparto non ammesso
	15	Numero file non valido su LEGGI
	16	Numero file su LEGGI non supportato dall'ECR
	17	L'ECR non supporta i gruppi
	18	Linea sorgente troppo lunga
	19	Numero ECR illegale
	20	Numero operatore non ammesso
2 (ERRORI DI I/O)	0	Errore di I/O (sia per comandi ECRCOM che EM_LINK)
3 (ERRORE CRITICO)	0	L'ECR segnala errore critico
4 (DATA ERROR)	0	L'ECR segnala data-error
5 (ERRORI ECR)	0	L'ECR segnala errore
6 (ERRORI BASE IN ESECUZIONE COMANDO EM_LINK)	0	Errore EM_LINK
	1	Si sono verificati errori durante l'esecuzione del file EM_LINK
	2	Comando EM_LINK errato o non ammesso
	3	Errore in apertura del file di input
	4	Errore in apertura del file degli errori
	5	Errore in apertura del file di log
	6	Errore in fase di azzeramento del file articoli
	7	Impossibile attivare stand-by su ECR
	8	Errore in apertura della sessione "Download Articoli" su ECR
	9	Fine imprevista della sessione "Download Articoli"
	10	Errato controllo del #records totali ricevuti in download
	11	File movimenti in overflow
	12	Impossibile attivare blocco. Transazione aperta su ECR
7 (ERRORI SINTASSI IN ESECUZIONE COMANDO EM_LINK)	0	Errore di sintassi
	1	Fine linea imprevisto
	2	Valore alfabetico non valido (apici)
	3	Codice articolo non valido
	4	Valore non numerico
	5	Valore iva non valido
	6	Reparto non valido
	7	Comando non valido

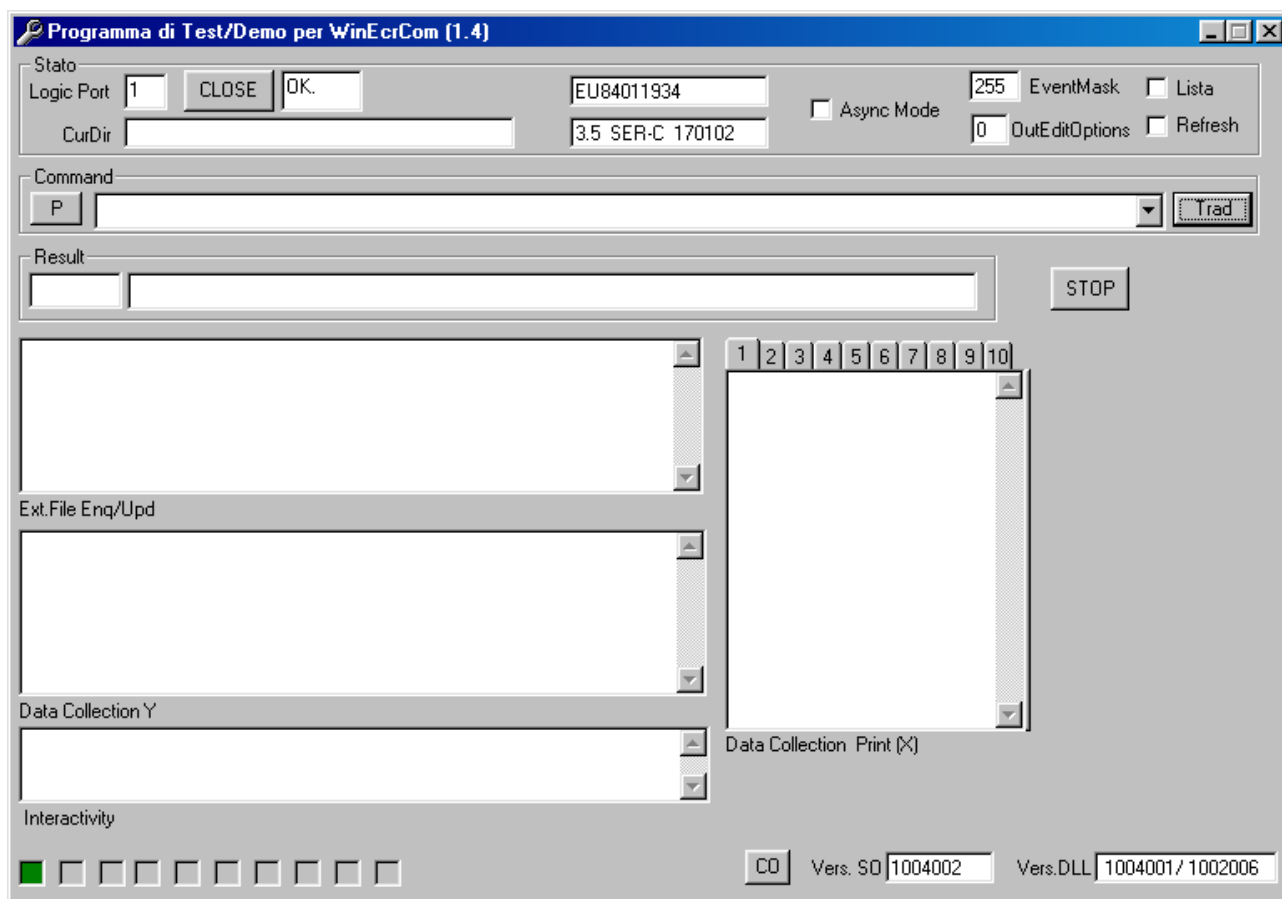
	8	Valore numerico non ammesso
8 (ERRORI ECR IN ESECUZIONE COMANDO EM_LINK)	0	L'ECR segnala errore
9 (ERRORE DI RISPOSTA INATTESA)	0	Errore di risposta inattesa

16. Esempio di utilizzo del “Programma di Test/Demo per WinEcrCom”

All'interno della cartella WinEcrCom\Demo\Vb è possibile trovare il file eseguibile “DemoVB.exe”. Esso è un programma di test che consente di effettuare una verifica di funzionamento del driver. La schermata principale è la seguente:



Inserendo nel campo “**Logic Port**” il valore relativo alla porta logica che si vuole utilizzare (ad es. 1) e cliccando sul pulsante “**Open**” viene mandato in esecuzione il Service Object. Se il collegamento è andato a buon fine si avrà:



quindi a fianco del pulsante “**Close**” comparirà “OK” ed inoltre saranno visualizzate la matricola del registratore di cassa (ad es. EU84011934) e la versione firmware (ad es. 3.5 SER-C 170102). Descriviamo nel dettaglio i restanti campi:

CurDir È il campo nel quale va inserita la directory corrente. Se non si inserisce nulla, per default sarà utilizzata la directory WinEcrCom\Demo\Vb.

N.B. Per posizionarsi nella directory voluta, inserire prima la CurDir e poi cliccare su “Open”

Async Mode Settando tale flag, si attiva la modalità di funzionamento asincrono.

EventMask Maschera degli eventi. È un valore che varia tra 0 e 255.

OutEditOptions Vedi il paragrafo “Proprietà”.

Lista Settando questa flag apparirà una lista che visualizzerà le linee di comando in Progress.

Refresh

Command In questa finestra si inseriscono i comandi da mandare in esecuzione.

Cliccando sul pulsante “P” è possibile visualizzare una serie di comandi preimpostati.

Trad Cliccando su tale pulsante, vengono mandati in esecuzione i comandi scritti nella finestra “Command”.

Result In tale finestra sarà visualizzato l’esito del comando mandato in esecuzione.

Stop Pulsante utilizzato per bloccare i comandi mandati in esecuzione.

Ext.File Enq/Upd Saranno visualizzati i dati di vendita degli articoli (nel caso in cui sull’ECR siano stati abilitati i “File Esterni”).

Data Collection Y Saranno visualizzati tutti i messaggi “Data Collect” (nel caso in cui essi siano stati abilitati sull’ECR).

Interactivity Saranno visualizzati tutti i messaggi di “Interattività” (nel caso in cui essi siano stati abilitati sull’ECR).

Data Collection Print (X) Saranno visualizzate tutte le righe stampate sullo scontrino (nel caso in cui sia stato abilitato sull’ECR il “Monitor di stampa”).

CO Richiama la versione del Control Object.

Vers. SO Visualizza la versione del Service Object.

Vers. DLL Visualizza la versione della DLL.

17. Output Model

L'output model del WinEcrCom è di due tipi:

sincrono ed *asincrono*. Una classe di dispositivi può supportare un tipo, entrambi o nessun tipo.

Modalità Sincrona

Tale modalità è preferita quando un comando può essere eseguito rapidamente. Il suo pregio è la semplicità.

L'applicativo chiama il metodo EcrCmd che torna solo quando il comando è stato completamente eseguito.

Modalità Asincrona

Tale modalità è preferita quando un comando richiede lente interazioni hardware. L'applicativo chiama il metodo EcrCmd che torna subito, ossia il comando viene eseguito dal driver in background e, nel frattempo, l'applicativo può eseguire altro codice. Quando il comando sarà stato completato, il driver invia un evento "CommandComplete" per segnalare all'applicativo l'avvenuta esecuzione e l'esito della esecuzione stessa. In tale modalità è quindi opportuno che l'evento "CommandComplete" sia abilitato.

La modalità asincrona è eseguita su una base *first-in first-out*.

18. Proprietà, metodi ed eventi

Proprietà

<i>Nome</i>		<i>Tipo</i>	<i>Accesso</i>
ControlObjectDescription	1.0	String	R
ControlObjectVersion	1.0	Long	R
EnableTradDC	1.0	Bool	
EventMask	1.0	Long	
OperatingMode	1.0	Long	
OutEditOptions	1.0	Long	
QuoteChar	1.0	Long	
ResultCode	1.0	Long	R
SeparatorChar	1.0	Long	
ServiceObjectVersion	1.0	Long	R
Status	1.0	Long	

Metodi

<i>Nome</i>		
AboutBox	1.0	Void
Close	1.0	Long
DirectIO	1.0	Long
EcrStatus	1.0	Long
EcrStatusEx	1.0	Long
EcrCmd	1.0	Long
EcrConfStr	1.0	String
EcrConfVal	1.0	Long
Open	1.0	Long

Eventi

<i>Nome</i>		
CommandComplete	1.0	Void
DataX	1.0	Void
DataY	1.0	Void
FileExtEnq	1.0	Void
FileExtUpd	1.0	Void
ProgressCtrl	1.0	Void
StatusChange	1.0	

18.1 Proprietà

ControlObjectDescription Property

Sintassi **BSTR ControlObjectDescription;**

Osservazioni Stringa che identifica il Control Object e l'azienda che lo produce.
La proprietà identifica il Control Object. Un esempio di stringa restituita è:
"CoEcrCom Control, Copyright(C) 2001 MicroPOS"
Tale proprietà è sempre leggibile.

Vedi anche **ControlObjectVersion** Property

ControlObjectVersion Property

Sintassi **LONG ControlObjectVersion;**

Osservazioni Numero di versione del Control Object.

Tale proprietà mantiene il numero di versione del Control Object. Tre livelli di versione sono specificati, come segue:

Livello di versione	Descrizione
Maggiore	Il posto "milioni". Un cambio al livello maggiore di versione WinEcrCom per una classe di dispositivi riflette significativi arricchimenti all'interfaccia e può rimuovere supporto per vecchie interfacce da precedenti livelli maggiori di versione.
Minore	Il posto "migliaia". Un cambio al livello minore di versione WinEcrCom per una classe di dispositivi riflette significativi arricchimenti all'interfaccia e deve fornire un set di precedenti interfacce a tale livello maggiore di versione.
Release	Il posto "unità". Livello interno fornito dallo sviluppatore di Control Object. Aggiornato quando sono fatte correzioni all'implementazione del CO.

Un esempio di numero di versione è:

1002038

Tale valore può essere visualizzato come versione "1.2.38" ed interpretato come versione maggiore 1, versione minore 2, release 38 del Control Object.

Tale proprietà è sempre leggibile.

Vedi anche **ControlObjectDescription** Property

Nota

Un Control Object per una classe di dispositivi opererà con ogni Service Object per quella classe, finchè il suo numero di versione maggiore corrisponde al numero di versione maggiore del Service Object. Se essi corrispondono, ma il numero di versione minore del Control Object è maggiore del numero di versione minore del Service Object, allora il Control Object può supportare alcuni nuovi metodi o proprietà che non sono supportati dalla release del Service Object. Le regole seguenti si applicano alle API supportate dalla release del Control Object ma non supportate dalla release più vecchia del Service Object:

- Leggendo una proprietà non supportata: il Control Object restituisce il valore non inizializzato della proprietà.
 - Scrivendo una proprietà non supportata: il Control Object ritorna, ma deve ricordare che una proprietà non supportata scrive o si è verificata una chiamata a metodi. Se l'applicazione legge la proprietà **ResultCode**, il Control Object deve restituire un valore di WEC_E_NOSERVICE (piuttosto che leggere il corrente **ResultCode** dal Service Object). **It must do this until the next property write or method call, at which time ResultCode is set by that API.**
 - Chiamando un metodo non supportato: il Control Object restituisce un v valore di WEC_E_NOSERVICE e deve ricordare che una proprietà non supportata scrive o si è verificata una chiamata a metodo. Se l'applicazione legge la proprietà **ResultCode**, il Control Object deve restituire un valore di WEC_E_NOSERVICE (piuttosto che leggere il corrente **ResultCode** dal Service Object). **It must do this until the next property write or method call, at which time ResultCode is set by that API.**
-

ResultCode Property

Sintassi **LONG ResultCode;**

Osservazioni Questa proprietà è settata da ogni metodo. Essa è anche settata quando è settata una proprietà scrivibile.

Questa proprietà è sempre leggibile. Prima che il metodo **Open** sia chiamato, essa ritorna il valore WEC_E_CLOSED.

I valori result code sono:

Valore	Significato
WEC_SUCCESS	Operazione con successo.
WEC_E_CLOSED	Si è tentato di accedere ad un dispositivo chiuso.
WEC_E_NOSERVICE	Il Control non può comunicare con il Service Object. Probabilmente un setup o una configurazione di errore dev'essere corretta.
WEC_E_DISABLED	Non può eseguire l'operazione mentre il dispositivo è disabilitato.
WEC_E_ILLEGAL	Si è tentato di eseguire un'operazione illegale o non supportata dal dispositivo, oppure è stato usato un parametro non valido.
WEC_E_NOHARDWARE	Il dispositivo non è connesso al sistema o non è alimentato.
WEC_E_OFFLINE	Il dispositivo è off-line.
WEC_E_NOEXIST	Il nome del file (o un altro valore specificato) non esiste.
WEC_E_EXISTS	Il nome del file (o un altro valore specificato) già esiste.
WEC_E_FAILURE	Il dispositivo non può eseguire la procedura richiesta, anche se esso è connesso al sistema, alimentato ed on-line.
WEC_E_TIMEOUT	Il Service Object è andato in time-out aspettando una risposta dal dispositivo, o il Control è andato in time-out aspettando una risposta dal Service Object.
WEC_E_BUSY	Lo stato corrente del Service Object non permette questa richiesta. Ad esempio, se l'uscita asincrona è in corso, alcuni metodi possono non essere permessi.
WEC_E_EXTENDED	E' accaduta una classe specifica di condizioni di errore. Il codice della condizione di errore è disponibile nella proprietà ResultCodeExtended .
Vedi anche	“Status”

ServiceObjectVersion Property

Sintassi**LONG ServiceObjectVersion;****Osservazioni**

Numero di versione del Service Object.

Questa proprietà mantiene il numero di versione del Service Object. Sono specificati tre livelli di versione, come segue:

Livello di versione	Descrizione
Maggiore	Il posto “milioni”. Un cambio al livello di versione maggiore WinEcrCom per una classe di dispositivi riflette significativi arricchimenti all’interfaccia e può rimuovere supporto per vecchie interfacce da precedenti livelli di versioni maggiore.
Minore	Il posto “migliaia”. Un cambio al livello di versione minore WinEcrCom per una classe di dispositivi riflette minori arricchimenti all’interfaccia e deve fornire un set di precedenti interfacce a questo maggiore livello di versione.
Release	Il posto “unità”. Livello interno fornito dallo sviluppatore di Service Object. Aggiornato quando sono fatte correzioni all’S.O.

Un esempio di numero versione è:

1002038

Questo valore può essere visualizzato come versione “1.2.38” ed interpretato come versione maggiore 1, versione minore 2, release 38 del Service Object.

Questa proprietà è inizializzata dal metodo **Open**.

Nota

Un Service Object per una classe di dispositivi opererà con un Control Object per quella classe, finché il suo numero di versione maggiore incontra il numero di versione maggiore del Control Object. Se essi si incontrano, ma il numero di versione minore del Service Object è maggiore del numero di versione minore del Control Object, allora il Service Object può supportare alcuni metodi o proprietà che non possono essere acceduti dalla release del Control Object. Se l’applicazione richiede tali caratteristiche, essa necessiterà di essere aggiornata per usare una versione passata del Control Object.

EnableTradDC Property

Sintassi **BOOL EnableTradDC;**

Osservazioni Tale proprietà abilita la traduzione dei messaggi data-collection Y

EventMask Property

Sintassi **LONG EventMask**

Osservazioni Gli eventi possono essere abilitati o meno a seconda di come viene impostata tale proprietà'. Il suo valore e' un numero dato dalla somma di vari "pesi". Ogni "peso" indica un particolare evento, secondo il seguente schema:

Data Collect X :	peso 1 (bit 0)
Data Collect Y :	peso 2 (bit 1)
External plu Enquiry :	peso 4 (bit 2)
External plu Update :	peso 8 (bit 3)
Interactivity :	peso 16 (bit 4)
Progress :	peso 32 (bit 5)
Command Complete :	peso 64 (bit 6)
StatusChange :	peso 128 (bit 7)

Se si vogliono tutti gli eventi, occorre passare :
 $1+2+4+8+16+32+64+128 = 255$

OperatingMode Property

Sintassi **LONG OperatingMode**

Osservazioni Tale proprietà influenza il modo di funzionamento del metodo EcrCmd, ossia del metodo che serve a inviare comandi al driver.

Se OperatingMode = 0 => modo "sincrono" ossia il metodo EcrCmd torna solo quando il comando è stato completamente eseguito.

Se OperatingMode = 1 => modo "asincrono" ossia il metodo EcrCmd torna subito, ossia il comando viene eseguito dal driver in back-ground e, nel frattempo, l'applicativo può eseguire altro codice. Quando il comando sarà stato completato, il driver invia un evento "CommandComplete" per segnalare all'applicativo l'avvenuta esecuzione e l'esito della esecuzione stessa. In tale modalità è quindi opportuno che l'evento CommandComplete sia abilitato.

OutEditOptions Property

Sintassi **LONG OutEditOptions;**

Osservazioni Tale proprietà controlla il modo in cui verranno in generale editati i campi sui files di output. Sono al momento definite le seguenti possibilità:

- 0 : Allinea i campi ed inserisce il carattere di separazione tra i campi
- 1 : Allinea i campi ma non inserisce il carattere di separazione
- 2 : Non allinea i campi ed inserisce il carattere di separazione tra i campi

QuoteChar Property

Sintassi **LONG QuoteChar;**

Osservazioni Tale proprietà consente di cambiare il carattere “apice” (ad es. presente nel campo descrizione del record articolo) in un altro carattere.

SeparatorChar Property

Sintassi **LONG SeparatorChar;**

Osservazioni Tale proprietà consente di cambiare il carattere “virgola” di separazione tra i vari campi (ad es. vend rep=1, pre=1) in un altro carattere.

Status Property

Sintassi **LONG Status;**

Osservazioni Tale proprietà ritorna lo stato del Service Object. Tale stato può essere:

Stato	Significato
IDLE	Stato di attesa (cioè può eseguire EcrCmd)
BUSY	Sta eseguendo un comando asincrono
CLOSED	Non è stata effettuata la Open
ERROR	Si è verificato un errore

18.2 Eventi

CommandComplete Event

Sintassi **VOID CommandComplete (LONG EcrNum, BSTR ResultString, LONG ResultCode);**

Osservazioni In caso di esecuzione di un comando in modo asincrono, si gestisce tale evento che viene inviato dall'SO quando il comando è stato completamente eseguito.

DataX Event

Sintassi **VOID DataX (LONG EcrNum, BSTR vString);**

Osservazioni È stato ricevuto un messaggio data-collect di tipo X. Visualizza il messaggio ricevuto sulla TextBox del canale.

DataY Event

Sintassi **VOID DataY (LONG EcrNum, BSTR vString);**

Osservazioni È stato ricevuto un messaggio data-collect di tipo Y.

FileExtEnq Event

Sintassi **VOID FileExtEnq (LONG EcrNum, LONG ArtType, BSTR ArtCode, DOUBLE Qty, LONG *pRecType, BSTR *pRecord) ;**

Osservazioni È stato ricevuto un messaggio di External File Enquiry. Questo evento si verifica quando un ecr invia un messaggio per richiedere i dati di vendita di un articolo. La routine di gestione di questo evento deve essenzialmente accedere ad un data-base e preparare una "risposta" con i dati dell'articolo.

Parametri di Input, ossia passati dall'SO :

EcrNum = Numero dell'ecr che ha inviato la richiesta

ArtType = Tipo di codice articolo:

0 = Numerico

1 = Alfanumerico

ArtCode = Stringa con il codice articolo da cercare

Qty = Quantità che l'ecr chiede di vendere

Parametri di Output, ossia che la routine DEVE restituire all'SO :

pRecType = Tipo Record. Indica all'SO il tipo di record che viene ad esso fornito.

0 = Tipo "normale"

1 = Tipo "con allegata offerta speciale"

pRecord = Stringa con il record-articolo

Nel caso di Tipo 0, questa deve essere composta dai seguenti campi separati da virgole :

<codice>,<descrizione>,<reparto>,<codice offerta>, <condizioni>,<prezzo 1>,<prezzo 2>,<prezzo 3>

FileExtUpd Event

Sintassi **VOID FileExtUpd (LONG EcrNum, LONG ArtType, BSTR ArtCode, DOUBLE Qty, DOUBLE Value) ;**

Osservazioni È stato ricevuto un messaggio di External File Update.

Questo evento si verifica quando un ecr invia un messaggio per richiedere al programma applicativo di aggiornare i dati di vendita.

Parametri di Input, ossia passati dall'SO :

EcrNum = Numero dell'ecr che ha inviato la richiesta

ArtType = Tipo di codice articolo :

0 = Numerico

1 = Alfanumerico

ArtCode = Stringa con il codice articolo da cercare

Qty = Quantità venduta

Value = Valore del venduto

ProgressCtrl Event

Sintassi **VOID ProgressCtrl (LONG EcrNum, LONG Count, LONG Total, BSTR vString, LONG *Abort);**

Osservazioni Questo evento viene notificato dall'SO durante la esecuzione di comandi di tipo batch ossia di files. Viene di norma inviato un evento "progress" ogni volta che viene eseguita una nuova linea del file sorgente.

Count = Se > 0, indica di norma il numero di linea sorgente.

Se Count > 100000, vuol dire che stiamo eseguendo un comando EM_LINK all'interno di un file comando.

In tal caso, Count-100000 = #linea sul file dati EM_LINK

Se = 0, ha un significato diverso : indica una segnalazione "interna"

Total = percentuale di avanzamento (tra 0 e 100)

StatusChange Event

Sintassi

Osservazioni Tale evento (disponibile solo a partire dalla versione 1.4) viene generato in caso di variazione dello stato del driver e/o di uno degli ecr connessi.

NumEcr = Numero dell'ecr il cui stato è variato

Code = Indica il tipo di variazione di stato avvenuta :

Al momento sono previsti i seguenti valori :

0 = uno degli ecr è stato sospeso/riattivato

Status = Indica un valore di stato.

Se l'evento riguarda un ecr, (Code=0) tale valore riporta i seguenti bit (pesi) :

peso 1 (bit 0) : 0 = ecr non presente 1 = ecr presente

peso 2 (bit 1) : 0 = ecr attivo 1 = ecr sospeso (non risponde)

peso 4 (bit 2) : U.F.

peso 8 (bit 3) : U.F.

peso 16 (bit 4) : U.F.

peso 32 (bit 5) : U.F.

peso 64 (bit 6) : U.F.

peso 128 (bit 7) : U.F.

Info = Stringa di testo contenente eventuali informazioni relative alla variazione avvenuta

In questo esempio, l'evento StatusChange viene utilizzato per visualizzare lo stato attuale degli ecr connessi in base al colore che appare sulle caselline di testo txtNetChange.

18.3 Metodi

AboutBox Method

Sintassi **VOID AboutBox ();**

Osservazioni Tale metodo visualizza le informazioni sul CO.

Close Method

Sintassi **LONG Close ();**

Osservazioni Chiamato per rilasciare il dispositivo e le sue risorse.

Restituzione Uno dei seguenti valori è restituito dal metodo e messo nella proprietà **ResultCode**:

Valore	Significato
WEC_SUCCESS	Il dispositivo è stato disabilitato e chiuso.
<i>Altri valori</i>	Vedi ResultCode .
Vedi anche	Open Method

DirectIO Method

Sintassi **LONG DirectIO (LONG, LONG*, BSTR*);**

Osservazioni Funzione da non utilizzare

EcrStatus Method

Sintassi **LONG EcrStatus (LONG ecrnum);**

Osservazioni Tale metodo fornisce lo stato in cui si trova l'ecr, vale a dire se è in idle generale, se è in transazione aperta, se è in errore critico, ecc.

EcrStatusEx Method

Sintassi **LONG EcrStatusEx (LONG ecrnum, BSTR* stato);**

Osservazioni Tale metodo è come quello EcrStatus, ma con delle informazioni aggiuntive (stato esteso).

EcrCmd Method

Sintassi **LONG EcrCmd (BSTR comando, BSTR* risposta);**

Osservazioni Tale metodo si usa per inviare all'SO un comando da eseguire. Esso ha due parametri:
Il primo è una stringa in cui c'è il comando in formato WinEcrCom che si vuole eseguire.
Il secondo è una stringa nella quale WinEcrCom passa una stringa che descrive l'esito della operazione. Il metodo EcrCmd torna un "long" che, se diverso da 0, indica il codice di errore che si è verificato. Il metodo EcrCmd torna "subito" nel caso sia stata attivata la modalità "asincrona". In tal caso il comando viene eseguito in background dall'SO il quale invierà un evento CommandComplete quando avrà completato la esecuzione. In caso di modalità "sincrona", invece se la EcrCmd torna 0, tutto è andato bene, altrimenti torna un codice di errore strutturato.

EcrConfStr Method

Sintassi **BSTR EcrConfStr (LONG valore);**

Osservazioni Tale metodo preleva e visualizza la matricola e la versione del registratore

correntemente selezionato, ossia il nodo 1, che è quello preselezionato di default al momento dell'apertura. Esso ha un parametro che permette di selezionare "quale" dato si vuole prelevare. Se il metodo EcrConfVal torna una stringa vuota, ciò indica di norma che non è stato possibile leggere la configurazione dell'ecr correntemente selezionato. Alcuni valori prelevabili sono:

EcrConfStr(0) = Stringa Stato corrente sul driver
EcrConfStr(1) = Matricola ecr selezionato
EcrConfStr(2) = Stringa-Modello ecr selezionato
EcrConfStr(3) = Stringa Versione Firmware ecr selezionato

EcrConfVal Method

Sintassi **LONG EcrConfVal (LONG valore);**

Osservazioni Il metodo EcrConfVal invece preleva dei valori numerici dalla configurazione dell'ecr selezionato. Anche EcrConfVal ha un parametro che specifica "quale" dato di configurazione deve essere letto. Se la EcrConfVal torna il valore -1, ciò indica che non è stato possibile leggere la configurazione dell'ecr correntemente selezionato. Alcuni valori prelevabili sono:

EcrConfVal(0) = Versione della DLL traduttore
EcrConfVal(1) = Versione della DLL protocollo
EcrConfVal(10) = Flag di configurazione valida. 0=non valida

Open Method

Sintassi **LONG Open (BSTR param);**

Il parametro *param* specifica il nome del dispositivo da aprire.

Osservazioni Chiamato per aprire un dispositivo per il successivo I/O.

Il nome del dispositivo specifica quale di uno o più dispositivi supportati da questo Control Object dovrebbe essere usato. Il *param* deve esistere nel registro di sistema per questa classe di dispositivi. La relazione tra il nome del dispositivo ed il dispositivo fisico è determinata dagli ingressi all'interno dell'**operating system registry**; questi ingressi sono mantenuti da un setup o da un'utilità di configurazione.

Quando il metodo **Open** ha successo, esso setta le proprietà **.....**, le descrizioni ed il numero di versione del WinEcrCom. **Additional class-specific properties may also be initialized.**

Restituzione Uno dei valori seguenti è restituito dal metodo:

Valore	Significato
WEC_SUCCESS	Aperto con successo.
WEC_E_ILLEGAL	Il Control è già aperto.
WEC_E_NOEXIST	Il <i>param</i> specificato non è stato trovato.
WEC_E_NOSERVICE	Potrebbe non stabilire una connessione con il corrispondente Service Object.
<i>Altri valori</i>	Vedi ResultCode .

Nota

Il valore della proprietà **ResultCode** dopo aver chiamato il metodo **Open** può non essere la stessa del valore restituito dal metodo **Open** per i seguenti due casi:

1. Il Control era chiuso ed il metodo **Open** ha fallito: la proprietà **ResultCode** continuerà a restituire WEC_E_CLOSED.
 2. Il Control era già aperto: il metodo **Open** restituirà WEC_E_ILLEGAL, ma la proprietà **ResultCode** può continuare a restituire il valore mantenuto prima del metodo **Open**.
-

Vedi anche **Close Method**